Лабораторная работа № 7

Тема: Управление оперативной памятью, виртуальной памятью и диагностика утечек ресурсов в Linux и Windows Server

Цель работы:

1. Научиться оценивать объемы и структуру использования памяти (физическая RAM, кэш, буферы, виртуальная память).
2. Освоить методы управления областью подкачки (Swap) в Linux через интерфейс командной строки.
3. Научиться выявлять процессы с утечками памяти и анализировать файлы подкачки (Pagefile) в Windows Server.

Используемое ПО: ВМ Linux (Debian/Ubuntu) и ВМ Windows Server.

ЧАСТЬ 1. РАБОТА В ОС LINUX (Анализ RAM и динамическое управление Swap)

Этап 1. Исследование состояния оперативной памяти

1. Получите общую статистику памяти

Выполните:

```
free -h
```

Параметр -h выводит значения в удобном для человека формате: MiB, GiB.

Зафиксируйте:

- total — общий объём физической RAM;
- used — память, используемая системой и процессами;
- free — полностью свободная память;
- buff/cache — память, используемая ядром под кэш и буферы;
- available — память, которую система может предоставить новым программам без серьёзной нехватки RAM;
- Swap — состояние области подкачки.

2. Получите подробную статистику памяти ядра

Выполните:

```
cat /proc/meminfo
```

Файл /proc/meminfo предоставляется ядром Linux и содержит подробную статистику использования памяти.

Сравните:

MemFree
и:
MemAvailable

Этап 2. Поиск процессов, использующих оперативную память
Теперь определите, какие процессы используют RAM.
1. Получите список процессов
Выполните:
ps aux --sort=-%mem | head
Команда ps aux — выводит список процессов;

- --sort=-%mem — сортирует их по использованию памяти от большего к меньшему;
- head — оставляет только первые строки.

Где:

- PID — идентификатор процесса;
- %MEM — процент физической памяти;
- RSS — сколько физической RAM фактически занимает процесс;
- COMMAND — выполняемая программа.

Определите три процесса, использующие больше всего памяти.

2. Исследуйте процессы в реальном времени
Выполните:
top
В окне top нажмите:
Shift + M
После этого процессы будут отсортированы по использованию памяти.

Для выхода из top нажмите:
q
Сравните результаты ps и top.

Этап 3. Исследование текущего Swap
1. Получите список активных областей подкачки
Выполните:
swapon --show
Команда показывает подключённые Swap-разделы и Swap-файлы.

Пример:
NAME TYPE SIZE USED PRIO
/dev/sda2 partition 1G 0B -2

Зафиксируйте:

- имя Swap;

- тип;
- размер;
- используемый объём.

2. Получите те же сведения через /proc
Выполните:
cat /proc/swaps
Сравните результат с:
swapon --show
Определите, отображают ли обе команды одинаковые области подкачки.

Этап 4. Создание дополнительного Swap-файла
Представьте ситуацию: на сервере недостаточно оперативной памяти, а существующий Swap мал. Не изменяя разметку диска, увеличьте Swap с помощью файла.
Это продолжает основное задание исходной лабораторной.

1. Создайте файл размером 1 ГБ
Выполните:
sudo dd if=/dev/zero of=/swapfile_extra bs=1M count=1024

Пояснение:
- if=/dev/zero — источник нулевых байтов;
- of=/swapfile_extra — создаваемый файл;
- bs=1M — размер блока 1 МБ;
- count=1024 — записать 1024 блока.

В результате будет создан файл примерно 1 ГБ.

2. Проверьте его размер
Выполните:
ls -lh /swapfile_extra
Убедитесь, что размер файла составляет примерно:
1.0G

3. Ограничьте права доступа
Выполните:
sudo chmod 600 /swapfile_extra
Затем:
ls -l /swapfile_extra
Ожидаемые права (Только владелец может читать и изменять файл):
-rw-------

Swap может содержать фрагменты памяти процессов, поэтому обычные пользователи не должны иметь к нему доступ.

4. Подготовьте файл как Swap
Выполните:
`sudo mkswap /swapfile_extra`
Команда создаёт внутри файла структуру области подкачки.

5. Подключите новый Swap
Выполните:
`sudo swapon /swapfile_extra`
Проверьте:
`swapon --show`
и:
`free -h`

Зафиксируйте новый общий объём Swap.

Этап 5. Контролируемое потребление оперативной памяти
Теперь исследуйте, как изменение нагрузки отражается на показателях памяти.
1. Зафиксируйте исходное состояние
Выполните:
`free -h`
Запишите:
- used;
- available;
- Swap used.

2. Запустите процесс, который выделит память
Выполните:
`python3 -c "import time; x=bytearray(300*1024*1024); time.sleep(300)"`
Команда:
- запускает Python;
- создаёт массив размером примерно 300 МБ;
- удерживает его в памяти около 5 минут.

Не закрывайте этот терминал.
Если у вас только один терминал добавьте в конец &, что бы процесс выполнялся в фоне:
`python3 -c "import time; x=bytearray(150*1024*1024); time.sleep(300)" &`

Этап 6. Анализ памяти во время нагрузки
Откройте второй терминал.
1. Получите новое состояние памяти
Выполните:
`free -h`

Сравните результат с исходным состоянием.
Определите:

- увеличился ли used;
- уменьшился ли available;
- изменилось ли использование Swap.

2. Найдите процесс, который создал нагрузку
Выполните:
ps aux --sort=-%mem | head
Найдите процесс python3.
Зафиксируйте:

- PID;
- %MEM;
- RSS.

Сравните RSS процесса с объёмом памяти, который вы указали в команде Python.

Этап 7. Наблюдение за памятью в реальном времени
Выполните:
watch -n 1 free -h
Команда watch будет выполнять free -h каждую секунду.
Оставьте окно открытым.
В другом терминале завершите созданный процесс:
kill PID
Например:
kill 2145
Если процесс уже завершился самостоятельно через 300 секунд, при необходимости запустите его повторно.
Наблюдайте, как после завершения процесса изменятся:

- used;
- available.

Для выхода из watch нажмите:
Ctrl + C

Этап 8. Исследование активности Swap
Наличие занятого Swap само по себе ещё не означает проблему. Важно определить, происходит ли постоянный обмен между RAM и Swap.
Выполните:
vmstat 1 5

Команда выполнит пять измерений с интервалом в одну секунду.
Обратите внимание на столбцы:

- free — свободная оперативная память;
- si — объём данных, переносимых из Swap в RAM;

- so — объём данных, переносимых из RAM в Swap.

Если в большинстве строк:
si = 0
so = 0
активного обмена со Swap в этот момент практически нет.
Зафиксируйте значения si и so.
Определите, происходил ли активный swapping во время измерения.

Этап 9. Проверка событий нехватки памяти
Linux может использовать механизм OOM Killer, если системе критически не хватает памяти.
Намеренно вызывать OOM в этой лабораторной не требуется.
Получите сообщения ядра:
journalctl -k | grep -i -E 'oom|out of memory'
Команда ищет сообщения, содержащие:
OOM
или:
Out of memory
Определите, происходили ли после загрузки системы события критической нехватки памяти.
Если команда ничего не вывела, зафиксируйте:
OOM-события не обнаружены.

Этап 10. Очистка учебного стенда
После выполнения лабораторной верните виртуальную машину в исходное состояние.
1. Отключите дополнительный Swap
sudo swapoff /swapfile_extra

2. Удалите Swap-файл
sudo rm /swapfile_extra

3. Проверьте состояние
swapon --show
и:
free -h
Убедитесь, что /swapfile_extra отсутствует, а система использует исходную конфигурацию Swap.

ЧАСТЬ 2. ДОПОЛНИТЕЛЬНОЕ ЗАДАНИЕ ДЛЯ WINDOWS SERVER (Диагностика виртуальной памяти)

В Windows Server задействуется механизм виртуальной памяти на базе файла pagefile.sys. Студенты научатся локализовать нехватку памяти и управлять этим файлом.

Этап 1: Мониторинг RAM и коммитов виртуальной памяти

1. Запустите ВМ Windows Server, откройте **Диспетчер задач** (Task Manager) ➔ вкладка **«Производительность»** (Performance) ➔ раздел **«Память»** (Memory).
2. Обратите внимание на параметр **«Выделено»** (Committed). Он отображается в виде дроби (например, 1.5/3.5 ГБ).
    - Первая цифра — это текущий объем виртуальной памяти, запрошенный программами.
    - Вторая цифра — это «Лимит коммита» (Сумма физической RAM + размер Pagefile).
3. Откройте **Монитор ресурсов** (Resource Monitor) через ссылку в Диспетчере задач. Перейдите на вкладку «Память». Изучите цветную полосу распределения памяти. Выпишите объем памяти, находящейся в состоянии «Ожидание» (Standby) — это аналог кэша в Linux.

Этап 2: Настройка и перенос файла подкачки (Pagefile)

Для повышения производительности серверов администраторы часто переносят файл подкачки с медленного системного диска на быстрый выделенный накопитель (в нашем случае — на созданный в ЛР №3 зеркальный диск E:).

1. Нажмите Win + R, введите sysdm.cpl (Свойства системы) ➔ вкладка **«Дополнительно»** (Advanced).
2. В разделе «Быстродействие» (Performance) нажмите **«Память» / «Параметры»** ➔ вкладка **«Дополнительно»** ➔ кнопка **«Изменить»** (Change) в разделе Виртуальная память.
3. Снимите галочку «Автоматически выбирать объем файла подкачки».
4. Выберите диск C:, установите переключатель **«Без файла подкачки»** (No paging file) и нажмите кнопку **«Задать»** (Set).
5. Выберите ваш диск E:, установите переключатель **«Размер по выбору системы»** (System managed size) или задайте вручную 2048 - 2048 МБ, и нажмите **«Задать»**.
6. Нажмите ОК. Система потребует **перезагрузку** для применения параметров. После перезагрузки убедитесь, что скрытый системный файл pagefile.sys исчез с диска C: и появился в корне диска E:.

***Дополнительное задание (Windows Server): Расследование инцидента истощения виртуальной памяти (REDR)**

Цель задания: 1. Научиться выявлять критическую нехватку оперативной памяти и лимита коммита (Commit Limit). 2. Спровоцировать контролируемое срабатывание подсистемы REDR. 3. Освоить аудит инцидентов истощения ресурсов с помощью Журнала событий (Event Viewer) по идентификаторам событий (Event ID).

Используемое ПО: ВМ Windows Server, утилита **Testlimit** (из официального пакета Microsoft Sysinternals).

ПОРЯДОК ВЫПОЛНЕНИЯ ЗАДАНИЯ:

Шаг 1. Подготовка и фиксация базового состояния

1. Запустите ВМ Windows Server и откройте **Диспетчер задач** (Ctrl+Shift+Esc) на вкладке **Производительность ➔ Память**.
2. Зафиксируйте в отчете текущие показатели:
    - Физическая память (в % и ГБ).
    - **Выделено (Committed):** запишите значение дроби (например, 1.2 / 2.5 ГБ). Напомним, что вторая цифра — это предел (физическая RAM + файл подкачки), при достижении которого Windows больше не сможет выдавать память.

Шаг 2. Скачивание инструмента симуляции утечки

Для симуляции атаки или сбойного приложения мы используем утилиту Testlimit, разработанную Марком Руссиновичем (главным архитектором Microsoft Azure). Она умеет безопасно «пожирать» память, не приводя к синему экрану (BSOD).

1. Откройте браузер на сервере и скачайте утилиту по прямой ссылке: https://live.sysinternals.com/testlimit64.exe (или скачайте пакет Sysinternals целиком).
2. Перенесите файл testlimit64.exe в удобную папку, например, в корень созданного ранее зеркального диска E:\.

Шаг 3. Искусственное истощение памяти и вызов REDR

1. Откройте командную строку (cmd) или PowerShell от имени Администратора.
2. Перейдите к папке с утилитой (например, cd E:\).
3. Запустите утилиту в режиме непрерывной утечки приватной памяти блоками по 100 Мегабайт:

```
testlimit64.exe -d 100 -c 10000
```

4. **Внимание!** Сразу после запуска переключитесь на Диспетчер задач. Вы увидите, как график «Выделено» (Committed) стремительно ползет вверх, приближаясь к лимиту, а система начнет ощутимо медленнее реагировать на действия.
5. Примерно через 15–30 секунд подсистема **REDR** обнаружит угрозу. На экране появится системное предупреждение: *«Закройте программы, чтобы предотвратить потерю данных»*.
6. Не закрывайте окно предупреждения вручную. Дождитесь, пока утилита testlimit64.exe завершится аварийно (Windows сама закроет процесс, когда Commit приблизится к 99%, а в консоли появится ошибка выделения памяти).

Шаг 4. Расследование инцидента в Журнале событий (Event Viewer)

Теперь, как системный администратор, вы должны провести «разбор полетов» и доказать руководству, какой именно процесс едва не уронил сервер.

1. Откройте утилиту **Просмотр событий** (Win + R ➔ eventvwr.msc).
2. Перейдите по дереву: **Журналы Windows** (Windows Logs) ➔ **Система** (System).
3. В правой панели нажмите кнопку **Фильтр текущего журнала** (Filter Current Log).
4. В поле «Источники событий» (Event sources) найдите и выберите: **Resource-Exhaustion-Detector**.
5. (Опционально) В поле ID событий введите: **2004**. Нажмите ОК.
6. Выберите последнее событие с кодом **2004** (Уровень: Предупреждение/Warning).
7. Перейдите на вкладку «Общие» и изучите текст.

Контрольный вопрос:

«Представьте, что на реальном продуктивном сервере под управлением Windows Server вы обнаружили в логах событие Event ID 2004, где главным потребителем памяти (Top Consumer) значится процесс sqlservr.exe (Microsoft SQL Server). Означает ли это, что сервер взломали, или это может быть штатным поведением СУБД? Каковы ваши действия как администратора?»

Правильный ответ: СУБД по умолчанию склонна забирать под кэш всю доступную RAM. Администратору нужно проверить настройки лимитов памяти (Max Server Memory) внутри самой SQL Server, чтобы она оставляла операционной системе жизненно важный запас.

ТРЕБОВАНИЯ К ОТЧЕТУ

1. **По разделу Linux:**
    - Скриншот вывода команды free -h до и после подключения дополнительного файла подкачки /swapfile_extra.
    - Скриншот успешного выполнения команд mkswap и swapon.
    - Письменный ответ на вопрос: *Почему операционная система Linux активно использует память под buff/cache, даже если пользователь не запускал тяжелых программ? Стоит ли сисадмину паниковать, если значение free близко к нулю, но available при этом составляет 50% от общего объема?*

2. **По разделу Windows Server:**
    - Скриншот окна «Виртуальная память» со сделанными настройками (перенос Pagefile на диск E:).
    - Скриншот вкладки «Память» Диспетчера задач после перезагрузки.
    - Скриншот Диспетчера задач в момент пиковой нагрузки (Шаг 3 Искусственное истощение памяти и вызов REDR), где видно, что строка «Выделено» (Committed) практически достигла своего максимума.

- Скриншот Журнала событий (Event Viewer) с открытым событием Event ID 2004. На скриншоте должны быть отмечены три ключевых параметра: общий объем запрошенной виртуальной памяти (Commit Charge) на момент сбоя? Какой процесс занял первое место в списке виновников (Top Consumers) и сколько конкретно байт/мегабайт он поглотил? (Студент должен увидеть там testlimit64.exe). Какой PID (идентификатор процесса) был у виновника?

Шпаргалка-памятка:

- **В Linux:** Если вы хотите сделать дополнительный Swap постоянным (чтобы он подгружался после перезагрузки ВМ), его строку нужно прописать в /etc/fstab, который мы изучали в ЛР №4, в формате: /swapfile_extra swap swap defaults 0 0. Но на этой лабораторной мы этого не делаем, чтобы не засорять систему.
- **В Windows Server:** Помните, что полностью отключать файл подкачки (Pagefile) крайне не рекомендуется, даже если на сервере очень много физической RAM. Некоторые системные службы ядра Windows и процессы генерации аварийных дампов памяти (Crash Dumps при BSOD) жестко завязаны на наличие файла подкачки.

1. Почему Linux активно забивает память под buff/cache?

С точки зрения ядра Linux, **пустая (неиспользуемая) оперативная память — это выброшенные на ветер деньги**. Физическая RAM работает в тысячи раз быстрее, чем самые современные NVMe SSD, не говоря уже о жестких дисках.

Когда операционная система считывает файлы с диска (например, системные библиотеки при загрузке, конфигурационные файлы, лог-файлы или код простых утилит), ядро не торопится удалять эти данные из памяти после завершения операции. Оно оставляет их в оперативной памяти в виде **кэша страниц (Page Cache)** и **буферов**.

Если системе или какой-то программе снова понадобятся эти файлы, Linux возьмет их прямо из сверхбыстрой RAM, вообще не обращаясь к медленному диску. Этот процесс происходит

постоянно и автоматически, поэтому даже без запуска «тяжелого» софта колонка buff/cache быстро разрастается практически на весь объем памяти.

2. Стоит ли паниковать, если free близко к нулю, а available равен 50%?

Короткий ответ: Абсолютно нет. Система находится в идеальном рабочем состоянии.

Вот почему важно различать эти параметры (их как раз выводят команды free -h или cat /proc/meminfo):

- **free (Абсолютно свободная память):** Это память, в которой вообще ничего нет — там записаны чистые нули. Ядро старается сводить этот показатель к минимуму, отдавая всё под кэш (buff/cache), как описано в первом пункте.
- **available (Доступная память):** Это реальное количество памяти, которое операционная система **может мгновенно отдать** под запуск новой программы или службы.

Что произойдет, если вы сейчас запустите тяжелое приложение? Ядру Linux не нужно проводить никаких сложных процедур. Оно мгновенно и безболезненно очистит (отнимет) нужный объем памяти из сектора buff/cache (ведь это просто копии файлов с диска, их можно стереть из памяти в любой момент без потери данных) и отдаст его вашему приложению.

🚨 **Когда сисадмину РЕАЛЬНО стоит паниковать?**

Паниковать нужно тогда, когда к нулю стремится показатель **available**, а наряду с этим:

1. Начинает активно и непрерывно расти использование **Swap (области подкачки)**.
2. Дисковая подсистема хоста загружается на 100% из-за постоянного сброса страниц памяти на диск (процесс Thrashing).
3. В логах (которые мы смотрим через journalctl) появляются записи от **OOM Killer** (Out-Of-Memory Killer) — механизма ядра, который начинает принудительно аварийно «убивать» процессы, чтобы спасти саму операционную систему от падения из-за полного исхода памяти.

Пока available равен 50%, ваш сервер чувствует себя отлично и использует аппаратные ресурсы на максимум для ускорения дисковых операций.